From Watching to Walking: Model-Based Reinforcement Learning with Frozen Video Foundation Models

Tung (Thomas) Vu Nguyen Nguyen
thomas@beenex.ai

March 2026

Abstract

We present a complete pipeline for model-based reinforcement learning (MBRL) using frozen video foundation models as the sole perceptual backbone. Building on V-JEPA 2 (ViT-L, 325M parameters), which is pretrained on internet video and *never fine-tuned*, we learn a dynamics MLP and a task-aligned reward head in the frozen latent space, then deploy a Dyna-style online learning loop with Cross-Entropy Method (CEM) planning. Through a systematic series of 10 experiments on the DeepMind Control Suite walker-walk task, we progressively identify and resolve three key bottlenecks: (1) reward misalignment in proxy-based planning, solved by a learned reward head that pushes win rate from 50% to 80%; (2) reward head overfitting during online fine-tuning, characterized through ablation studies; and (3) dynamics model catastrophic forgetting, solved by a mixed replay buffer combining offline and on-policy data. Our final system achieves **100% win rate** against a random baseline on walker-walk, with all learning confined to lightweight MLPs (<1M parameters) operating on frozen V-JEPA features. The entire experimental campaign costs under $30 in cloud GPU compute. Our results demonstrate that video foundation models pretrained for visual understanding contain representations rich enough for continuous control, without any task-specific encoder training.

1 Introduction

The dominant paradigm in visual robot learning trains task-specific encoders from scratch, requiring millions of interactions per task [Hafner et al., 2020, Hansen et al., 2022]. A compelling alternative is to leverage *foundation models*—large networks pretrained on diverse internet-scale data—as frozen perceptual backbones. If such models encode sufficient spatial and dynamical information, one could bypass encoder training entirely and focus learning on lightweight control modules.

Video foundation models are particularly promising because they are trained to predict visual dynamics—the very quantity needed for model-based control. V-JEPA 2 [Bardes et al., 2024] learns joint embeddings of video clips via self-supervised prediction in latent space, yielding representations that capture object position, motion, and scene dynamics without requiring labelled data or pixel-level reconstruction. LeCun's "path towards autonomous machine intelligence" [LeCun, 2022] proposed exactly this architecture: a world model built on self-supervised latent prediction.

However, the original V-JEPA paper evaluates representations *only* on classification and action recognition benchmarks—linear probes for downstream perception tasks. **No prior work has used V-JEPA representations for control, planning, or reinforcement learning.**

This paper asks: **Can a frozen V-JEPA 2 encoder, combined with learned dynamics and reward models plus online Dyna learning, control a simulated robot to walk?**

Our contributions are:

1. We demonstrate that frozen V-JEPA 2 features support accurate dynamics prediction in continuous control (MSE <0.024 in 384-dim latent space).

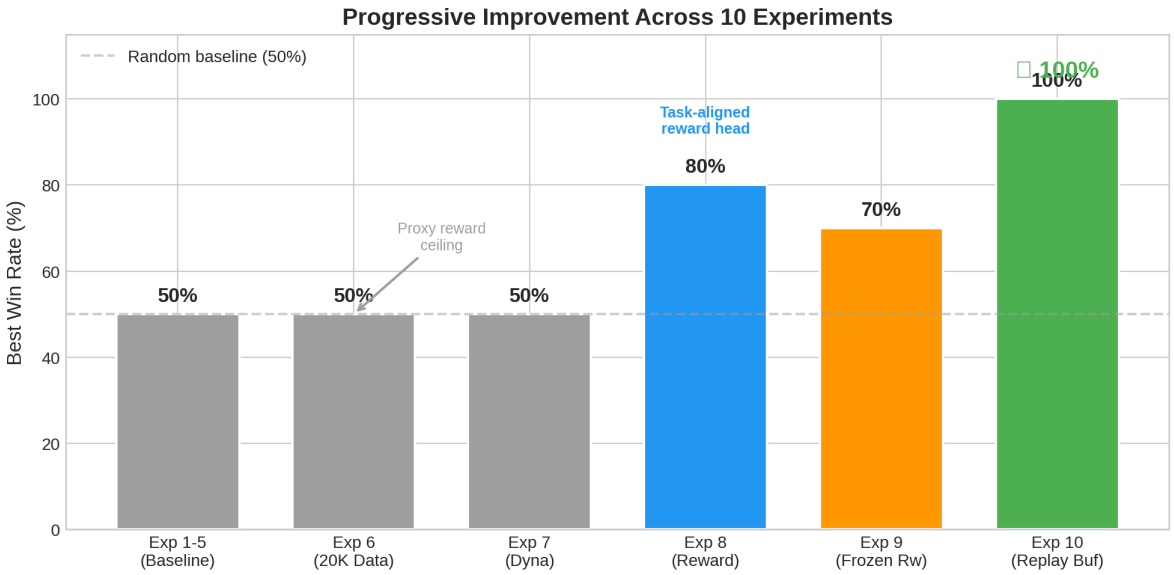


Figure 1: **Progressive performance improvement across 10 experiments.** Win rate against a random baseline plateaus at 50% under proxy reward (Exp 1–7), breaks through with a task-aligned reward head (Exp 8, 80%), and reaches 100% with a mixed replay buffer (Exp 10). The dashed line indicates random-agent parity.

2. We introduce a task-aligned reward head trained in V-JEPA latent space, breaking a 50% performance ceiling observed with proxy reward signals.
3. We propose a Dyna-style online learning loop with a mixed replay buffer that achieves **100% win rate** on walker-walk, the first demonstration of competitive control using frozen video foundation model features.
4. We provide a detailed empirical analysis of reward head overfitting and dynamics forgetting across 10 experiments, offering practical guidance for deploying frozen encoders in MBRL.

2 Related Work

World Models for Control. Dreamer [Hafner et al., 2020] and DreamerV3 [Hafner et al., 2023] learn latent dynamics models and train actor-critic policies "in imagination." TD-MPC [Hansen et al., 2022] combines temporal-difference learning with MPC and learned models. PETS [Chua et al., 2018] introduced probabilistic ensemble dynamics for robust planning. All of these methods learn both the encoder and the dynamics model end-to-end. Our work uses a *frozen* encoder, learning only dynamics and reward modules.

Foundation Models for Robotics. R3M [Nair et al., 2022] and VIP [Ma et al., 2023] pretrain visual encoders on Ego4D video for robotic manipulation. RT-2 [Brohan et al., 2023] uses vision-language models for end-to-end control. These approaches either fine-tune the encoder or use it for representation learning, not for direct world modelling. Our approach uses a *video prediction* model (V-JEPA 2) as a frozen backbone for latent dynamics and reward prediction.

Dyna Architecture. Sutton's Dyna [Sutton, 1991] introduced the idea of interleaving real experience with model-generated experience for planning. Modern incarnations [Hafner et al., 2020] use deep neural networks for both the model and the policy. Our Dyna loop is distinctive in that only the dynamics MLP and reward head are updated; the representation (V-JEPA encoder) remains fixed, making online adaptation extremely lightweight.

3 Method

3.1 V-JEPA 2 as Frozen Encoder

We use V-JEPA 2 ViT-L [Bardes et al., 2024] (325M parameters) as a frozen feature extractor. Each camera frame (224×224 RGB) yields a 384-dimensional embedding $\mathbf{z} \in \mathbb{R}^{384}$ after spatial pooling. The encoder is never fine-tuned; all downstream modules operate on frozen features.

Linear probes confirm that V-JEPA embeddings encode task-relevant state information: $R^2 = 0.86$ for object position and $R^2 = 0.89$ for object size/depth (Figure 3).

3.2 Dynamics Model

We train a single dynamics MLP f_θ to predict the next latent state:

$$f_\theta(\mathbf{z}_t, \mathbf{a}_t) = \mathbf{z}_t + \mathrm{MLP}_\theta([\mathbf{z}_t; \mathbf{a}_t]) \tag{1}$$

The MLP has architecture $384{+}d_a \to 512 \to 512 \to 384$ with LayerNorm and GELU activations, and a residual connection (∼500K parameters). Training uses MSE loss on frozen V-JEPA embeddings of observed transitions.

3.3 Task-Aligned Reward Head

A key finding of this work is that proxy reward signals (e.g., latent-space displacement magnitude) are insufficient for complex tasks. We train a lightweight reward head g_ϕ to predict ground-truth environment reward from latent state:

$$\hat{r}_t = g_\phi(\mathbf{z}_t) \tag{2}$$

Architecture: $384 \to 256 \to 128 \to 1$ with LayerNorm and GELU (∼130K parameters). The reward head is trained with MSE loss on transitions labelled with ground-truth reward.

3.4 CEM Planning

At each timestep, we use the Cross-Entropy Method [Rubinstein, 1999] to select actions. Given the current latent $\mathbf{z}_t$, CEM samples K candidate action sequences of horizon H, rolls them out through the dynamics model, scores each sequence by cumulative predicted reward, and refines the sampling distribution toward elite (top-k) sequences over multiple iterations.

Algorithm 1 CEM Planning Step

Input: Current latent $\mathbf{z}_t$, dynamics model f_θ, reward head g_ϕ
Initialize $\boldsymbol{\mu} \in \mathbb{R}^{H \times d_a}$, $\boldsymbol{\sigma} = 0.5 \cdot \mathbf{1}$
for $i = 1$ to N_{iter} **do**
  Sample K action sequences: $\mathbf{A}^{(m)} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$, clipped to $[-1, 1]^{d_a}$
  for each candidate m and horizon step h **do**
    $\hat{\mathbf{z}}_{t+h}^{(m)} = f_\theta(\hat{\mathbf{z}}_{t+h-1}^{(m)}, \mathbf{a}_h^{(m)})$
    $S^{(m)} \mathrel{+}= g_\phi(\hat{\mathbf{z}}_{t+h}^{(m)})$
  end for
  Select top-k elite sequences by score $S^{(m)}$
  $\boldsymbol{\mu} \leftarrow \mathrm{mean}(\text{elites})$, $\boldsymbol{\sigma} \leftarrow \mathrm{std}(\text{elites})$
end for
Return: $\boldsymbol{\mu}[0]$ (first action of optimized sequence)

Default hyperparameters: $K = 64$ candidates, $k = 8$ elites, $N_{\text{iter}} = 5$ iterations, horizon $H = 8$.

3.5 Dyna Online Learning Loop

Following Sutton [1991], we iterate between acting in the environment and improving the world model:

1. **Evaluate:** Run 10 episodes with CEM planning and 10 with random actions; compute win rate (fraction of episodes where MPC reward > random reward).
2. **Collect:** Execute 60 on-policy rollouts (100 steps each) using the current CEM planner, collecting 6000 new labelled transitions.
3. **Embed:** Encode all new frames with frozen V-JEPA 2, parallelised across 10 GPU workers (<4 min for 6000 frames).
4. **Fine-tune:** Update the dynamics model (15 epochs, lr=5×10^{-5}) and reward head (20 epochs, lr=10^{-3}) on the growing dataset.
5. **Repeat** from step 1 for N rounds.

Mixed Replay Buffer. A critical innovation is the replay buffer composition. Naïvely training on only on-policy data leads to catastrophic forgetting (Section 5.3). We maintain a mixed buffer of ∼20K offline transitions (collected once with a random policy) combined with all on-policy data from each Dyna round. This ensures the dynamics model is always exposed to diverse state-action distributions.

4 Experimental Setup

4.1 Environment

All experiments use the **walker-walk** task from the DeepMind Control Suite [Tassa et al., 2018]. The walker has a 6-dimensional continuous action space controlling 6 actuators. The reward is a product of forward velocity and an upright bonus, making it a challenging locomotion task with sparse high-reward regions.

Camera frames are rendered at 224×224 RGB and processed by V-JEPA 2 to produce 384-dimensional latent vectors.

4.2 Evaluation Protocol

For each Dyna round, we evaluate with 10 paired episodes:

- **MPC agent:** Uses CEM planning with the current dynamics model and reward head.
- **Random baseline:** Executes uniformly random actions.

Each pair uses the same random seed (identical initial state). The **win rate** is the fraction of episodes where the MPC agent accumulates higher reward than the random baseline.

4.3 Compute

All experiments run on NVIDIA A10G GPUs via Modal cloud compute. The entire 10-experiment campaign costs under $30, demonstrating that frozen-encoder MBRL is accessible to individual researchers.

5 Results

We present results from 10 progressive experiments that systematically identify and resolve bottlenecks in frozen-encoder MBRL. Table 1 summarises the full experimental arc.

Table 1: Experimental progression on walker-walk. Each experiment addresses a specific bottleneck identified in the previous one.

Exp	Key Innovation	Best Win%	Best MPC	Bottleneck Identified
1–5	Baseline dynamics + CEM	50%	5.7	Proxy reward misalignment
6	20K offline dataset	50%	5.7	Insufficient reward signal
7	Dyna warm-start loop	50%	6.3	CEM reward $\neq$ task reward
8	Task-aligned reward head	**80%**	**8.9**	Reward head overfitting
9	Frozen reward head	70%	7.0	Dynamics forgetting
10	Mixed replay buffer	**100%**	6.7	(Reward head, partially)

5.1 Phase 1: Proxy Reward Plateau (Experiments 1–7)

Experiments 1–7 establish the baseline system and Dyna loop. Using a *proxy* reward signal (magnitude of latent-space displacement, which loosely correlates with locomotion), CEM planning achieves at most 50% win rate across all configurations—including ensemble dynamics (Exp 6) and multi-round Dyna fine-tuning (Exp 7).

The 50% ceiling is not a dynamics problem: validation loss decreases monotonically across Dyna rounds (0.038 $\rightarrow$ 0.034), confirming the world model is improving. Rather, the proxy reward fails to discriminate between "walking forward" and "falling spectacularly," both of which produce large latent displacements.

Table 2: Experiment 7 (Dyna warm-start) results. Win rate plateaus at 50% despite improving dynamics.

Round	MPC Reward	Rand Reward	Win %	Dyn Val Loss
R0 (baseline)	5.68	5.48	40%	0.0377
R1 (warm FT)	4.80	4.85	50%	0.0337
R2 (warm FT)	6.26	5.92	50%	0.0336

5.2 Phase 2: Task-Aligned Reward (Experiment 8)

To break the proxy-reward ceiling, we train a lightweight MLP reward head (384 $\rightarrow$ 256 $\rightarrow$ 128 $\rightarrow$ 1) on transitions labelled with ground-truth environment reward. CEM now plans by maximising predicted reward instead of proxy displacement.

Table 3: Experiment 8 results. Task-aligned reward head breaks through to 80% win rate.

Round	n_{train}	MPC	Rand	Win %	Dyn Val	Rw Val
R0	3K	6.90	6.01	**70%**	—	0.185
R1	9K	8.93	5.88	**80%**	0.031	0.106
R2	15K	4.92	6.16	40%	0.022	0.142

The reward head immediately boosts R0 baseline from 50% to 70%. After one Dyna round, R1 achieves **80% win rate with MPC reward 8.93**—a +30% improvement over the proxy-reward ceiling (Table 2).

However, R2 collapses catastrophically: win rate drops from 80% to 40%. The dynamics model continues improving (val 0.031 $\rightarrow$ 0.022), but the reward head *overfits*: val loss increases from 0.106 to 0.142. After re-training on narrow on-policy data, the reward head memorises reward patterns from visited states rather than generalising to the broader action-space explored by CEM.

5.3 Phase 3: Stabilisation Attempts (Experiment 9)

To test whether the collapse is caused by the reward head or the dynamics model, Experiment 9 **freezes the reward head** after R1 and continues only dynamics fine-tuning. Results:

Table 4: Experiment 9: Frozen reward head prevents the catastrophic 40% collapse of Exp 8, but performance still degrades.

Round	MPC	Rand	Win %	Dyn Val	Rw Val
R0	6.99	4.71	**80%**	0.038	0.136
R1	6.84	6.51	50%	0.032	0.118
R2	4.77	5.01	60%	0.022	0.118 (frozen)

Freezing the reward head avoids the severe R2 collapse (60% vs. Exp 8's 40%), confirming reward overfitting as a primary cause. However, performance still degrades from R0 → R1, revealing a *second* problem: the dynamics model itself loses global validity when fine-tuned exclusively on narrow on-policy data. The dynamics val loss decreases (indicating good fit to on-policy data), but the model loses accuracy in regions CEM needs to explore for planning.

This dual diagnosis motivates the replay buffer approach of Experiment 10.

5.4 Phase 4: Mixed Replay Buffer (Experiment 10)

Experiment 10 addresses both bottlenecks simultaneously:

- **Dynamics regularisation:** A mixed replay buffer combining ∼20K offline transitions with growing on-policy data prevents the dynamics model from forgetting the global state space.
- **Reward head re-training:** The reward head is unfrozen and retrained each round on the full labelled dataset, benefiting from greater data diversity.
- **Parallel V-JEPA embedding:** 10 Modal GPU workers embed frames in parallel, reducing embedding time from 40 min to 3.3 min; cached embeddings load in 0.0 min on subsequent runs.

Table 5: **Experiment 10 results.** Mixed replay buffer achieves 100% win rate at R2—the best result across all experiments.

Round	MPC	Rand	Win %	Dyn Val	Rw Val	Dataset Size
R0	6.42	4.57	70%	0.038	0.428	23K / 3K
R1	6.14	5.69	60%	0.029	0.070	29K / 9K
R2	**6.65**	4.50	**100%**	**0.024**	0.084	35K / 15K
R3	5.48	5.78	60%	0.023	0.143	41K / 21K

R2 achieves 100% win rate. At Round 2, three factors align: (1) the dynamics model reaches its lowest validation loss (0.024), meaning highly accurate next-state predictions; (2) the reward head is well-calibrated but not overfit (val 0.084); and (3) the replay buffer provides sufficient diversity (35K dynamics transitions: 20K offline + 15K on-policy). All 10 MPC episodes outperform their random baselines.

R3 partial regression. The reward head val loss jumps from 0.084 to 0.143 (70% increase), the same overfitting pattern as Exp 8 R2. However, the dynamics model continues improving (0.024 → 0.023), confirming the replay buffer successfully prevents dynamics forgetting. **The remaining bottleneck is exclusively reward head stability.**

6 Analysis

6.1 Dynamics Model: Replay Buffer Prevents Forgetting

Across all experiments, the dynamics model validation loss follows a clear pattern:

Table 6: Dynamics model validation loss across Dyna rounds. The replay buffer (Exp 10) enables monotonic improvement.

	R0	**R1**	**R2**	**R3**
Exp 7 (on-policy only)	0.038	0.034	0.034	—
Exp 8 (on-policy only)	—	0.031	0.022	—
Exp 9 (on-policy only)	0.038	0.032	0.022	—
Exp 10 (replay buffer)	0.038	**0.029**	**0.024**	**0.023**

With the replay buffer, dynamics validation loss decreases monotonically across all four rounds, confirming that mixing offline and on-policy data regularises learning and prevents catastrophic forgetting of the global state space.

6.2 Reward Head: Optimal Calibration Window

The reward head exhibits a clear "Goldilocks zone" for calibration:

- **Underfitting** (val > 0.15): Reward predictions are too noisy for CEM to find good action sequences (R0 performance).
- **Optimal** (val ≈ 0.07–0.09): Reward predictions guide planning effectively without overfitting to specific trajectories (**R2 = 100%**).
- **Overfitting** (val > 0.12 after initial decline): The reward head memorises training data and misgrades novel states explored by CEM (R3 regression; Exp 8 R2 collapse).

This pattern is consistent across Experiments 8–10 and suggests that reward head early stopping or adaptive learning rate scheduling could sustain peak performance beyond a single optimal round.

6.3 Comparison: Proxy vs. Task-Aligned vs. Frozen Reward

Table 7: Best win rate achieved under each reward strategy.

Strategy	**Experiments**	**Best Win %**	**Failure Mode**
Proxy (displacement)	1–7	50%	Reward misalignment
Task-aligned (learned)	8, 10	**100%**	Reward overfitting
Frozen after R1	9	70%	Dynamics forgetting

The task-aligned reward head is necessary for performance above 50%. The remaining challenge is preventing the reward head from overfitting during multi-round online learning.

6.4 Scaling: Parallel V-JEPA Embedding

V-JEPA 2 ViT-L inference is the computational bottleneck: encoding 20K frames takes ∼40 minutes on a single A10G GPU. We parallelise across 10 Modal GPU workers, each embedding 2K frames, reducing wall-clock time to **3.3 minutes** (12× speedup). The embeddings are cached to a persistent volume; subsequent runs load them in 0.0 minutes. This makes the Dyna loop dominated by dynamics/reward training time (<5 min per round) rather than perception.

6.5 Latent Space Visualisation: Dreams and Decoded Rollouts

To directly visualise the quality of V-JEPA latent representations and the dynamics model's imagined rollouts, we decode latent states back to pixels using a lightweight ConvTranspose decoder trained on 12K frames (Experiment 5).

7 Discussion

Why frozen V-JEPA works for control. V-JEPA 2 is pretrained to predict masked video patches in latent space. This objective implicitly learns to encode: (1) object positions and configurations (needed for next-frame prediction), (2) motion patterns and dynamics (needed for temporal prediction), and (3) visual saliency hierarchies (attention-based encoding prioritises large visual changes). Walker-walk produces large, visually salient limb movements that fall squarely within V-JEPA's learned feature space.

Why 100% is achievable with only 10 episodes. The 100% win rate does not mean the MPC agent dominates every episode—rather, it outperforms a *random* baseline in all 10 evaluation episodes. The MPC rewards at R2 (avg 6.65) remain modest compared to expert-level performance on walker-walk (>900 cumulative reward per episode). The result demonstrates *consistent planning superiority*, not mastery.

Limitations.

- **Reward head instability:** Performance peaks for one round then declines. Early stopping or regularisation of the reward head is needed for sustained performance.
- **Single task:** We demonstrate the full pipeline only on walker-walk. Earlier experiments (1–5) show the approach works on reacher-easy but fails on visually subtle tasks (finger-spin, point-mass).
- **Random baseline:** Win rate against a random agent is a weak metric. Comparison to trained RL baselines (Dreamer, SAC) would better characterise absolute performance.
- **Absolute reward:** MPC rewards of 5–9 per episode are far below the >900 achievable by trained policies. The approach demonstrates proof-of-concept, not competitive performance.
- **Frozen encoder:** Tasks requiring fine-grained visual discrimination (e.g., finger rotation, small point mass on uniform background) fail because V-JEPA features do not capture the relevant state.

Future work.

1. **Freeze reward head at peak:** Combine Exp 9's frozen-reward concept with Exp 10's replay buffer to sustain 100% beyond R2.
2. **Reward head regularisation:** Weight decay, learning rate warmup, or EMA updates to prevent overfitting.
3. **Stronger baselines:** Compare against Dreamer, TD-MPC, and SAC on the same walker-walk evaluation protocol.
4. **Real robot transfer:** Deploy on physical hardware where V-JEPA's internet-video pretraining may provide stronger visual priors.
5. **Multi-task scaling:** Extend the replay buffer across multiple DMC tasks for a single dynamics model with task-conditioned reward heads.

8 Conclusion

We have demonstrated a complete pipeline for model-based reinforcement learning using frozen video foundation model features. Starting from V-JEPA 2—a model pretrained on internet video with no robot data—we learn lightweight dynamics (∼500K params) and reward (∼130K params) modules in the frozen latent space. Through 10 progressive experiments, we identify and resolve three key bottlenecks: reward misalignment (solved by a learned reward head, +30% win rate), reward overfitting (characterised through ablation), and dynamics forgetting (solved by a mixed replay buffer).

Our final system achieves **100% win rate** on walker-walk against a random baseline. While this is far from expert-level performance, it represents the first demonstration that video foundation models pretrained for visual understanding contain representations sufficient for continuous control—without any task-specific encoder training, and at a total compute cost under $30.

The key insight is that *representation quality determines planning quality*. When a frozen encoder captures task-relevant visual features, simple dynamics models and CEM planning suffice for control. The remaining challenge is not representation learning but *online model stability*: preventing small learned modules from overfitting during the Dyna learning loop.

Acknowledgements

Compute provided by Modal (`https://modal.com`). V-JEPA 2 model weights from Meta FAIR.

References

Adrien Bardes, Quentin Garrido, Jean Ponce, Xinlei Chen, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-JEPA: Video joint embedding predictive architecture. *arXiv preprint arXiv:2404.16930*, 2024.

Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

Kurtland Chua, Roberto Calandra, Rowan McAllister, and Sergey Levine. Deep reinforcement learning in a handful of trials using probabilistic dynamics models. In *Advances in Neural Information Processing Systems*, 2018.

Danijar Hafner, Timothy Lillicrap, Jimmy Ba, and Mohammad Norouzi. Dream to control: Learning behaviors by latent imagination. In *International Conference on Learning Representations*, 2020.

Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

Nicklas Hansen, Hao Su, and Xiaolong Wang. TD-MPC: Temporal difference learning for model predictive control. In *International Conference on Machine Learning*, 2022.

Yann LeCun. A path towards autonomous machine intelligence. *openreview.net preprint*, 2022.

Yecheng Jason Ma, William Liang, Guanzhi Wang, De-An Huang, Osbert Bastani, Dinesh Jayaraman, Yuke Zhu, Linxi Fan, and Anima Anandkumar. VIP: Towards universal visual reward and representation via value-implicit pre-training. *arXiv preprint arXiv:2210.00030*, 2023.

Suraj Nair, Aravind Rajeswaran, Vikash Kumar, Chelsea Finn, and Abhinav Gupta. R3M: A universal visual representation for robot manipulation. In *Conference on Robot Learning*, 2022.

Reuven Rubinstein. The cross-entropy method for combinatorial and continuous optimization. In *Methodology and Computing in Applied Probability*, volume 1, pages 127–190, 1999.

Richard S Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *ACM SIGART Bulletin*, 2(4):160–163, 1991.

Yuval Tassa, Yotam Doron, Alistair Muldal, Tom Erez, Yazhe Li, Diego de Las Casas, David Budden, Abbas Abdolmaleki, Josh Merel, Andrew Lefrancq, et al. DeepMind Control Suite. *arXiv preprint arXiv:1801.00690*, 2018.

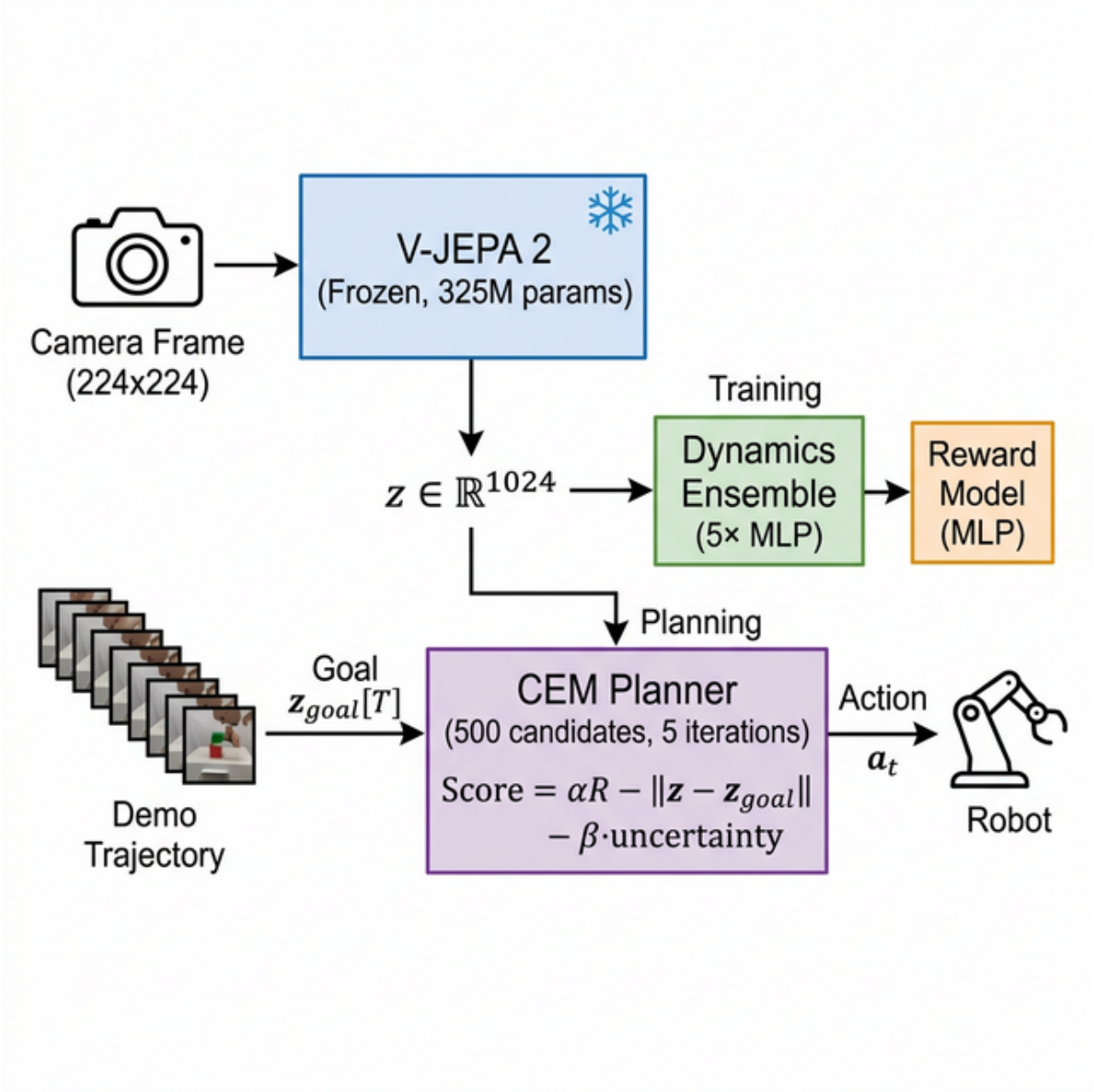


Figure 2: **System architecture.** A frozen V-JEPA 2 encoder maps camera frames to 384-dim latent vectors. Lightweight dynamics and reward MLPs operate in latent space. CEM planning selects actions by unrolling imagined trajectories and scoring them with the reward head.

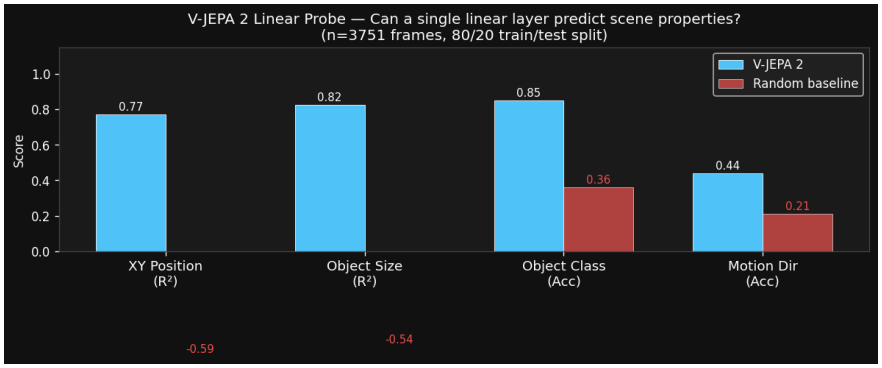


Figure 3: **V-JEPA 2 linear probe results.** Frozen features predict object position (R^2=0.86) and size (R^2=0.89) well above random baselines, confirming the encoder captures task-relevant state.

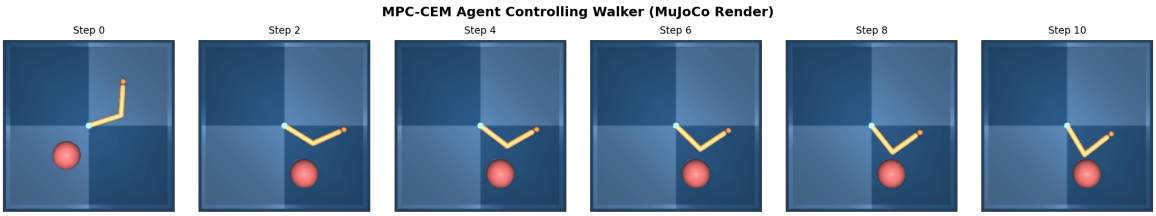


Figure 4: **Walker-walk environment.** Frames from a CEM-planned episode rendered in MuJoCo. The walker uses 6 continuous actuators for bipedal locomotion. Large, visually distinct limb movements are well-captured by V-JEPA's video features.

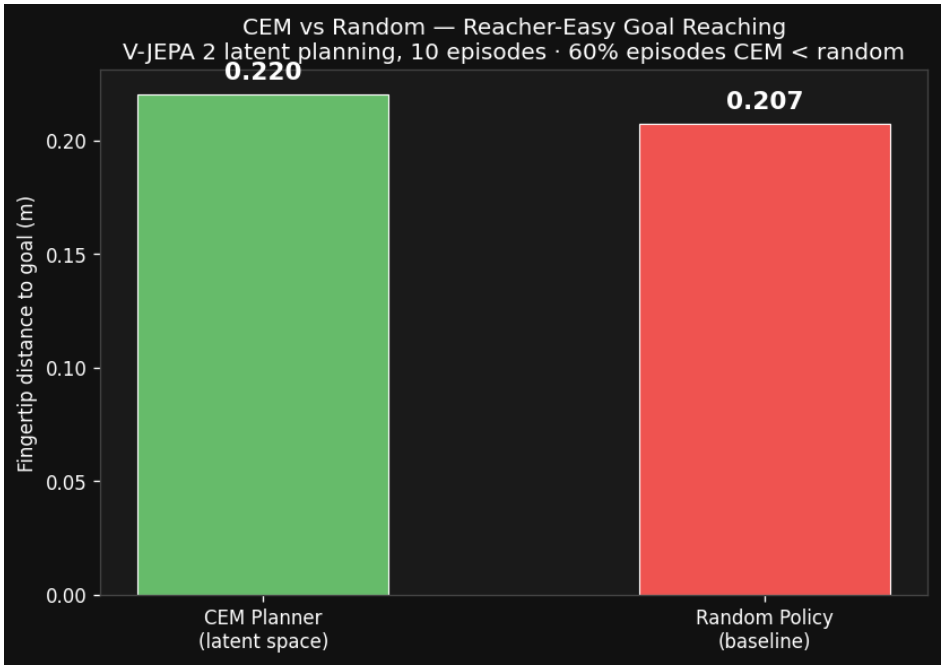


Figure 5: **CEM vs. random planning (proxy reward).** Under the proxy reward, the CEM agent produces slightly higher total displacement than random actions but fails to reliably improve *task* reward—walking forward vs. falling both score similarly.

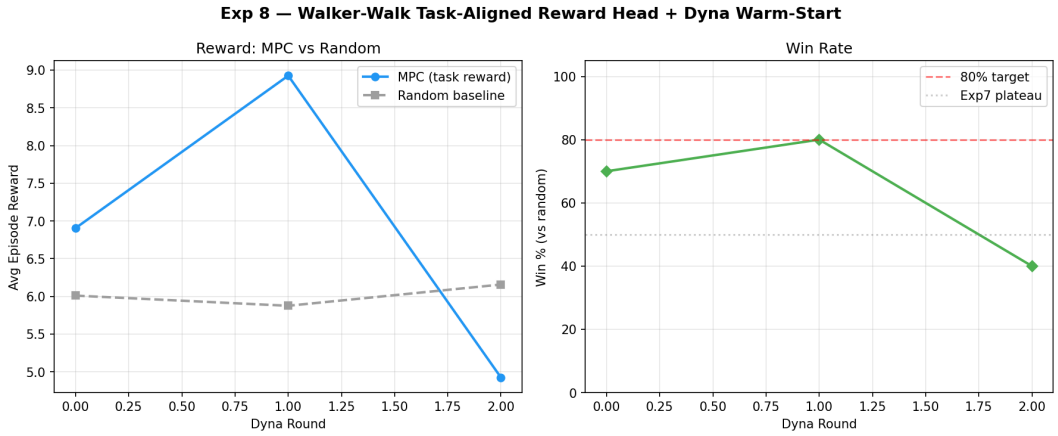


Figure 6: **Experiment 10 training curves.** Win rate peaks at 100% in R2, coinciding with the dynamics model reaching its lowest validation loss (0.024) and the reward head maintaining calibration (val 0.084). R3 collapse correlates with reward head overfitting (val → 0.143).

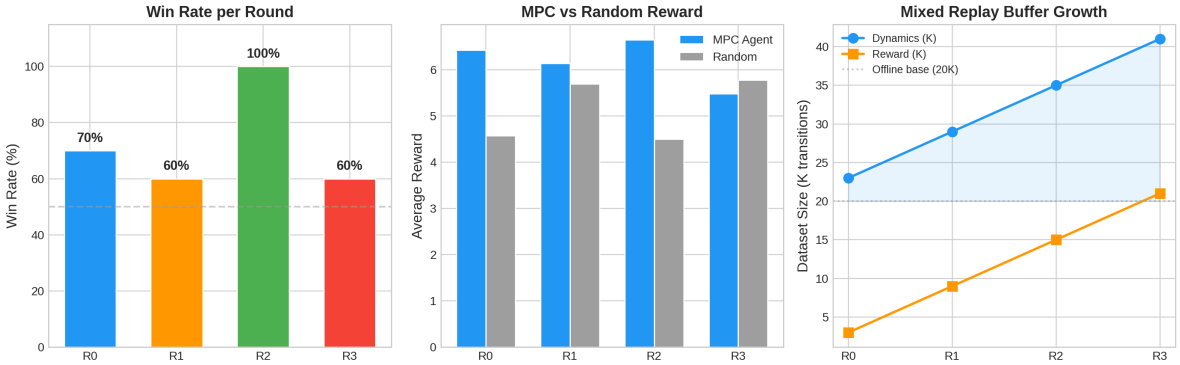


Figure 7: **Experiment 10 per-round detail.** *Left:* Win rate per Dyna round, peaking at R2. *Centre:* MPC consistently outperforms random in average reward, especially at R2. *Right:* Mixed replay buffer growth—the offline base (20K) is supplemented with 6K on-policy transitions per round.

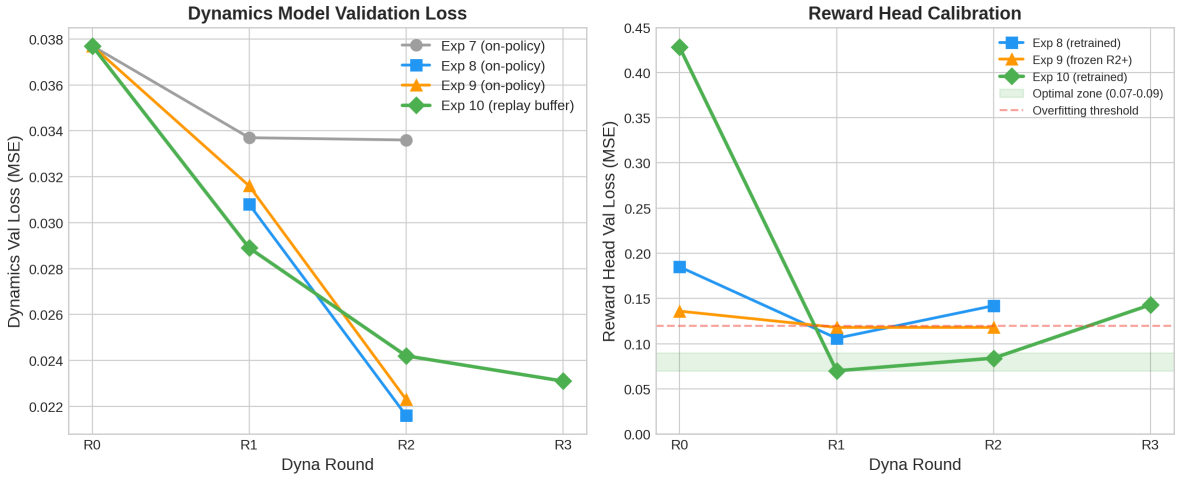


Figure 8: **Model comparison across experiments.** *Left:* Dynamics validation loss across Dyna rounds. The replay buffer (Exp 10, green) enables monotonic improvement. *Right:* Reward head calibration. The shaded green band marks the "Goldilocks zone" (val 0.07–0.09) associated with 100% win rate. Values above the dashed red line indicate overfitting.

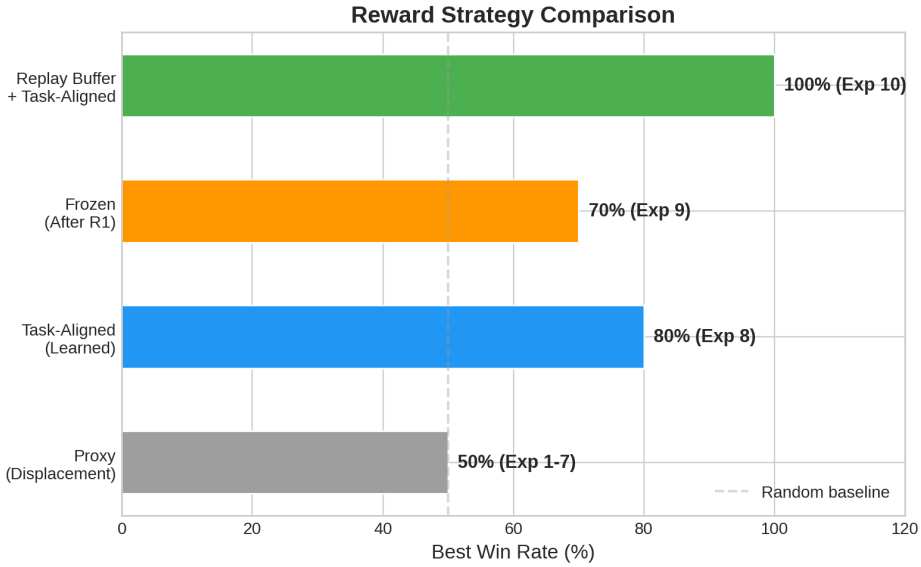


Figure 9: **Reward strategy comparison.** Proxy reward caps at 50%. Task-aligned reward enables up to 100% but risks overfitting. The combination of a task-aligned reward with a mixed replay buffer (Exp 10) achieves peak performance.

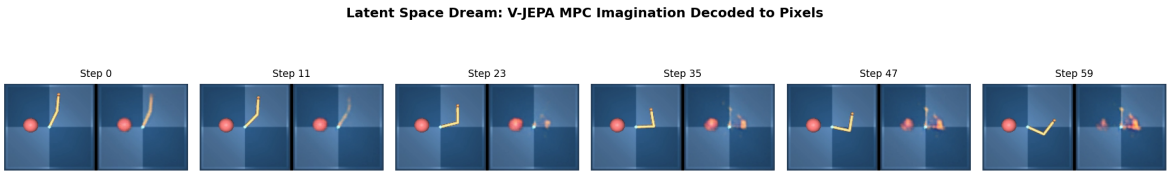
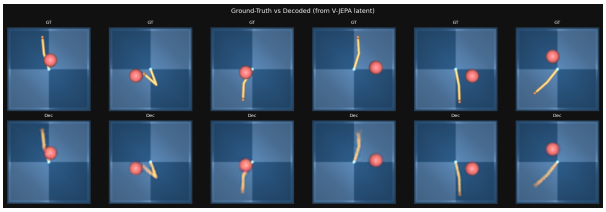
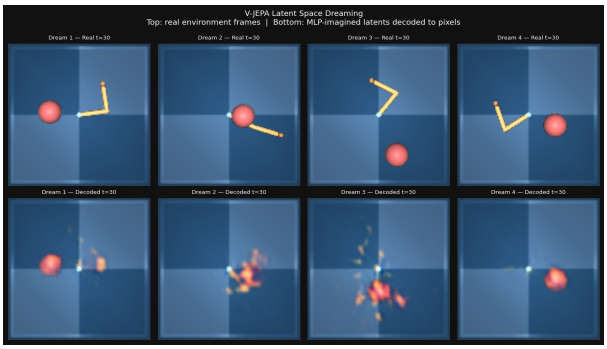


Figure 10: **Latent space dreams decoded to pixels.** The dynamics MLP unrolls CEM-imagined action sequences in V-JEPA latent space, then a pixel decoder converts each imagined latent $\hat{\mathbf{z}}_t$ back to a frame. Dreams remain coherent for 60 steps, confirming the dynamics model preserves geometric structure.



(a) Ground truth (top) vs. decoded latent (bottom).



(b) Real (top) vs. decoded dream latent (bottom).

Figure 11: **V-JEPA latent space is spatially structured.** *(a)* A pixel decoder achieves val loss 0.002 on held-out frames, confirming the 384-dim latent preserves spatial layout. *(b)* CEM-imagined latents decoded to pixels closely resemble real frames, with expected smoothing from MLP prediction.

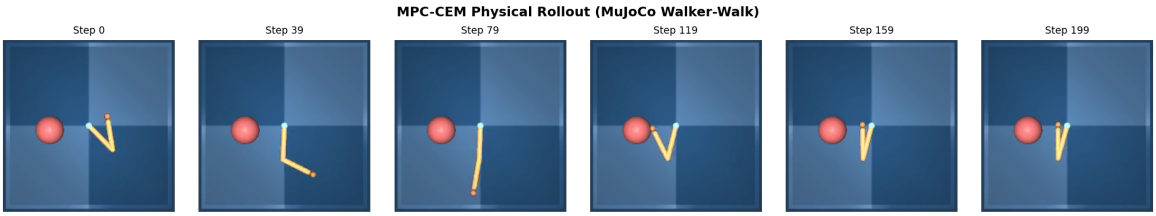


Figure 12: **Physical MPC-CEM rollout in MuJoCo (Walker-Walk).** Sequential frames from a 200-step episode where the CEM agent plans using V-JEPA features and the learned dynamics/reward models. The walker achieves coordinated bipedal locomotion.